

NAAN MUDHALAVAN

PROJECT REPORT

AI STUDY BUDDY API

A Study Assistance System Using Artificial Intelligence and REST API

Prepared for Academic Project Submission

Academic Year: 2026–2027

S.NO	TEAM	NAME	EMAIL ID
1	Team Leader	Thanuja Saraswathi S	thanujasaraswathineet2023@gmail.com
2	Team Member	Nabeesha A	ansarabdulwahab3344@gmail.com
3	Team Member	Nivetha S	rajapan715@gmail.com
4	Team Member	Parameshwari S	parameshwari7221@gmail.com
5	Team Member	Preethi M	shobisuha1607@gmail.com

1. ABSTRACT

AI Study Buddy API is an intelligent educational application programming interface designed to provide students with personalized study assistance. The system uses Artificial Intelligence techniques to understand student questions, generate explanations, summarize learning material, create quizzes, and support revision activities. Instead of building separate AI functionality for every application, the proposed system exposes these services through REST API endpoints so that web and mobile applications can communicate with the AI study engine.

The system is designed to make learning more interactive and accessible. A student can submit a question or learning topic through an API request and receive a structured response. Additional services can include text summarization, question generation, quiz evaluation, study-plan generation, and basic progress tracking. The API-based architecture makes the system reusable, scalable, and suitable for integration with different educational interfaces.

The main goal of the project is to provide a digital study companion that supports students during self-learning. The system does not replace teachers or textbooks; rather, it provides quick assistance and organizes learning activities in a convenient way.

2. INTRODUCTION

Artificial Intelligence is increasingly being used in education to provide personalized learning experiences. Traditional learning systems generally present the same material to every student, while students may have different levels of understanding, learning speeds, and revision requirements. An AI-based study assistant can respond to individual learning needs by providing explanations and practice activities.

An API, or Application Programming Interface, allows two software systems to communicate. In this project, the AI Study Buddy API acts as a bridge between a user interface and AI-powered educational services. A frontend application can send a request containing a student's question, topic, or study material. The backend processes the request and returns an appropriate response in a structured format such as JSON.

The project combines REST API concepts, backend programming, database management, and Artificial Intelligence. It can be implemented using Python with a web framework such as Flask or FastAPI. A database such as SQLite or MySQL can store user information, questions, quiz results, and study history.

3. PROBLEM STATEMENT

Students often spend significant time searching multiple sources for explanations, revision questions, summaries, and study plans. Generic online resources may not always provide answers in a student's preferred level or format. There is therefore a need for a centralized study assistant that can provide quick and structured learning support.

- Students may have difficulty obtaining immediate explanations for doubts.
- Revision material may be scattered across different sources.
- Creating practice questions manually can be time-consuming.
- Students may need personalized study schedules and reminders.
- Different applications may require the same AI study features.
- A reusable API can provide a common backend for web and mobile clients.

4. OBJECTIVES

- To develop an AI-powered study assistance API.
- To provide question-answering support for educational topics.
- To generate concise summaries from study material.
- To generate practice questions and quizzes.
- To evaluate quiz answers and provide feedback.
- To support personalized study-plan generation.
- To store relevant study history and progress.
- To expose AI features through secure REST API endpoints.
- To create a backend that can be integrated with different frontend applications.

5. EXISTING SYSTEM

In the existing approach, students commonly depend on textbooks, search engines, educational websites, video platforms, and manually prepared notes. These resources are useful but may require students to search through multiple sources. Most basic learning applications provide fixed content and do not dynamically generate explanations or practice material based on the student's request.

Another limitation is that different educational applications may implement similar functionality separately. This increases development effort and makes maintenance more difficult. A common API layer can reduce duplication by providing reusable AI services.

6. PROPOSED SYSTEM

The proposed AI Study Buddy API provides a centralized backend for AI-assisted learning. The system receives requests from an authorized client, validates the request, processes the educational task, communicates with an AI model when required, and returns a structured response.

The architecture separates the frontend from the AI and data-processing logic. This allows the same API to be used by a web application, Android application, desktop application, or other educational software.

- Question and answer assistance
- Study material summarization
- Topic explanation
- Quiz and practice-question generation
- Answer evaluation and feedback
- Personalized study-plan generation
- Study history and progress tracking
- REST/JSON based communication

7. SYSTEM REQUIREMENTS

7.1 Hardware Requirements

- Processor: Dual-core processor or above
- RAM: Minimum 4 GB; 8 GB recommended
- Storage: At least 5 GB free space for development
- Internet connection for cloud AI model/API access

7.2 Software Requirements

- Operating System: Windows, Linux, or macOS
- Python 3.x
- Flask or FastAPI
- SQLite/MySQL
- REST API client such as Postman
- Web browser
- AI model/API provider compatible with the backend

8. SYSTEM ARCHITECTURE

The system follows a client-server architecture. The client sends an HTTP request to the API server. The API server validates authentication and request data, forwards the educational task to the appropriate service, communicates with the AI model if required, optionally stores the result in the database, and returns a JSON response.

The major layers are: Presentation Layer, API Layer, AI Service Layer, Database Layer, and External AI Model/API Layer.

- Client Layer – Web or mobile application used by students.
- API Layer – Handles HTTP requests, authentication, validation, and routing.
- AI Service Layer – Processes prompts and educational tasks.
- Database Layer – Stores users, study sessions, quizzes, and results.
- AI Model Layer – Provides language understanding and generation capabilities.

9. MODULES

9.1 User Management Module

This module manages user registration, login, authentication, and basic profile information. Authentication tokens can be used so that only authorized users can access protected endpoints.

9.2 Ask Study Buddy Module

Students can submit a question or topic. The backend prepares the request and sends it to the AI service. The response is returned as a clear explanation. The application can optionally accept difficulty level or preferred answer length.

9.3 Summarization Module

The student can provide study material as text. The system generates a shorter summary containing the important points. This can help students during revision.

9.4 Quiz Generation Module

The system can generate multiple-choice, true/false, or short-answer questions from a selected topic. Questions can be generated according to difficulty level.

9.5 Quiz Evaluation Module

Submitted answers are evaluated by the backend. Objective questions can be evaluated using stored answers, while suitable short-answer tasks may use AI-assisted comparison. The result can include score and feedback.

9.6 Study Plan Module

The system can create a study plan based on topics, available study time, target date, and preferred schedule. The plan divides topics into manageable study sessions.

9.7 Progress Tracking Module

The application stores quiz scores and completed study sessions. Students can use this information to understand their study activity and identify topics requiring further practice.

10. API DESIGN

The API follows REST principles and commonly uses HTTP methods such as GET, POST, PUT, and DELETE. Data is exchanged using JSON.

Endpoint	Method	Purpose	Example Request
/api/login	POST	Authenticate user	username, password
/api/ask	POST	Ask a study question	question, topic
/api/summarize	POST	Summarize study material	text
/api/quiz/generate	POST	Generate quiz	topic, difficulty
/api/quiz/evaluate	POST	Evaluate quiz answers	quiz_id, answers
/api/study-plan	POST	Create study plan	subjects, date
/api/progress	GET	View progress	user token

11. SAMPLE JSON RESPONSE

A typical question-answer response may be returned in the following structure:

```
{
  "success": true,
  "topic": "Operating System",
  "question": "What is a process?",
  "answer": "A process is a program that is currently being executed.",
  "difficulty": "basic"
}
```

12. DATABASE DESIGN

A relational database can be used to store application data. The following tables are suitable for the proposed system.

Table	Important Fields	Purpose
users	id, name, email, password_hash	User accounts
study_sessions	id, user_id, topic, created_at	Study activity
questions	id, user_id, question, answer	Questions and responses
quizzes	id, user_id, topic, difficulty	Generated quizzes
quiz_results	id, quiz_id, score, submitted_at	Quiz performance
study_plans	id, user_id, plan_data, target_date	Personalized plans

13. WORKING PROCESS

- The student opens the frontend application.
- The student logs in or creates an account.
- The client sends an authenticated API request.
- The API validates the request and input data.
- The appropriate AI study module is selected.
- The AI service prepares a suitable prompt or processing task.
- The AI model generates or analyzes the educational content.
- The backend validates and formats the response.
- Relevant activity is stored in the database.
- The JSON response is sent back to the client.
- The frontend displays the result to the student.

14. SECURITY CONSIDERATIONS

Security is important because the API may handle user accounts and study history. Passwords should never be stored as plain text; secure password hashing should be used. Authentication tokens should be protected and transmitted over HTTPS in production. API input should be validated to reduce invalid requests and abuse.

- Use password hashing.
- Use HTTPS in production.
- Validate and sanitize user input.
- Protect API keys and AI service credentials using environment variables.
- Use authentication and authorization for protected endpoints.
- Apply rate limiting where appropriate.
- Avoid storing unnecessary personal information.

15. TESTING

Testing verifies that each API function works as expected. API testing can be performed using Postman or another REST client.

Test Case	Expected Result	Status
Valid login	Authentication succeeds	Pass
Invalid login	Error response returned	Pass
Ask question	AI answer returned in JSON	Pass
Empty question	Validation error returned	Pass
Generate quiz	Questions generated for topic	Pass
Evaluate quiz	Score and feedback returned	Pass
View progress	User progress returned	Pass

16. ADVANTAGES

- Provides quick educational assistance.
- Can personalize explanations according to user requirements.
- Reduces the need to build separate AI logic in every frontend.
- Can be integrated with web and mobile applications.
- Supports interactive learning through quizzes and feedback.
- Allows study activity and progress to be organized.
- REST API architecture supports future expansion.

17. LIMITATIONS

- AI-generated answers may occasionally contain errors.
- Internet access may be required for cloud AI services.
- AI API usage can involve cost depending on the provider.
- Response quality depends on the model and input quality.
- The system should not be treated as a replacement for teachers or official academic resources.

18. FUTURE ENHANCEMENTS

- Voice-based question answering.
- Support for multiple Indian languages.
- PDF and document-based question answering.
- Automatic flashcard generation.
- Spaced-repetition revision system.
- Learning analytics dashboard.
- Mobile application integration.
- Teacher dashboard for classroom-level analytics.
- Offline support for selected study resources.
- Integration with educational platforms.

19. CONCLUSION

AI Study Buddy API is a practical application of Artificial Intelligence, REST API development, and database management in the education domain. The system provides a reusable backend that can deliver question answering, summarization, quiz generation, evaluation, study planning, and progress tracking. Its API-based architecture allows different client applications to use the same educational services.

The project demonstrates how AI can be combined with conventional software engineering techniques to create an interactive learning support system. With appropriate validation, security, and human oversight, the system can be extended into a larger educational platform.

20. REFERENCES

- Python Documentation – Official Python programming language documentation.
- Flask Documentation – Documentation for the Flask web framework.
- FastAPI Documentation – Documentation for modern Python API development.
- SQLite Documentation – Official SQLite database documentation.
- MDN Web Docs – HTTP, REST, JSON, and web development references.
- OpenAI API Documentation or documentation of the selected AI model provider.